Laboratorio 1 - Cálculo Diferencial

Mathias Salazar Salazar
Iván Fernando Carlosama
Samuel Teran
Diana Marcela Ocampo

Sesión 1: la ciudad en una tabla.

Conociendo el dataset:

```
1 print("Filas y columnas:", df.shape)
2 print(df.head(10))   # las primeras 10 filas, para ver la cara de los datos
3 df.tail()
```

```
Filas y columnas: (113440, 3)
      hogar      mes     kwh
0  MAC000002  2012-11  345.02
1  MAC000002  2012-12  355.76
2  MAC000002  2013-01  360.08
3  MAC000002  2013-02  381.84
4  MAC000002  2013-03  480.17
5  MAC000002  2013-04  362.74
6  MAC000002  2013-05  304.06
7  MAC000002  2013-06  300.12
8  MAC000002  2013-07  271.32
9  MAC000002  2013-08  270.05
```

	hogar	mes	kwh
113435	MAC005567	2013-10	158.54
113436	MAC005567	2013-11	173.09
113437	MAC005567	2013-12	186.35
113438	MAC005567	2014-01	171.12
113439	MAC005567	2014-02	139.04

Este primer pedazo de código nos muestra a través de los primeros 10 datos según su fecha de publicación, esto gracias a df.head(10), que, si no tuviera ese 10, toma los primeros 5 de manera predeterminada, como se puede ver con df.tail, que toma los últimos 5 datos según su fecha de publicación igualmente.

```python
# 🖍️ Ejercicio 1 — Completen las dos líneas que faltan.
print("Hogares distintos:", df["hogar"].nunique())
print("Primer mes:", df["mes"].min())
print("Último mes:", df["mes"].max())                    # ← su turno
print("¿Cuántos meses distintos hay?:", df["mes"].nunique())   # ← su turno
```

```
Hogares distintos: 5556
Primer mes: 2011-12
Último mes: 2014-02
¿Cuántos meses distintos hay?: 27
```

Aca simplemente en las 2 últimas líneas de código se editan algunos valores de las 2 primeras líneas que se imprimen, recalcando el método .nunique, que se encarga de retornar todos los valores distintos de una columna, gracias a la librería de pandas.

```python
# 🖍️ Ejercicio 2 — Juzgar y decidir.
# Si la policía de dominios encontró infractores (NaN, negativos, duplicados, valores absurdos),
# decidan QUÉ HACER con cada tipo (eliminar, corregir, conservar con nota) y háganlo aquí.
# Regla de oro: toda decisión queda escrita en un comentario. Un dato eliminado sin explicación
# es un dato escondido.

# df = df[ ... ]   # ← su decisión, si aplica
print("Filas después de la limpieza:", len(df))
```

```
Filas después de la limpieza: 113440
```

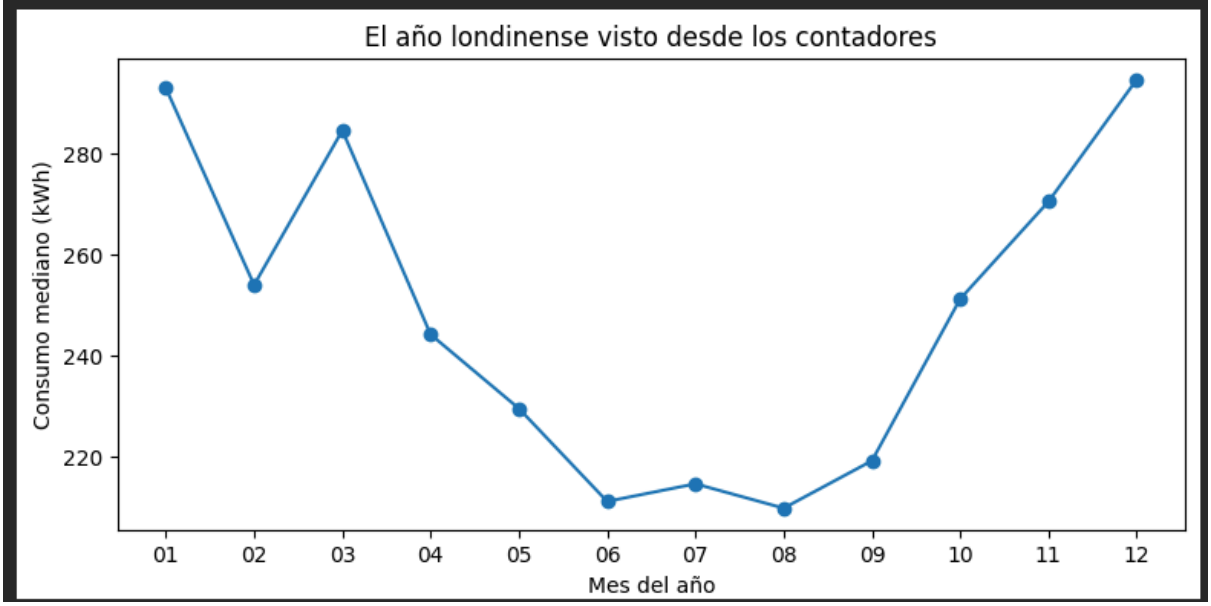
En este ejercicio no se añade nada de lo que se pide, ya que anteriormente no se encontraron valores absurdos.

```python
# 🖍️ Ejercicio 3 — El invierno en una curva.
# Construyan la curva del consumo MEDIANO por mes calendario (ene, feb, ..., dic).
# Pista: extraigan el número de mes con  df["mes"].str[5:7]  y usen groupby + median.

df["mes_num"] = df["mes"].str[5:7]
mediana_mensual = df.groupby("mes_num")["kwh"].median()

plt.figure(figsize=(9, 4))
plt.plot(mediana_mensual.index, mediana_mensual.values, marker="o")
plt.xlabel("Mes del año")
plt.ylabel("Consumo mediano (kWh)")
plt.title("El año londinense visto desde los contadores")
plt.show()

# 🗣️ ¿Apareció el invierno donde lo predijeron en P2? ¿En qué meses? ¿Por qué el pico
#    está donde está? (Piensen en la calefacción... y en qué hemisferio queda Londres.)
```



Con esta gráfica nos dimos cuenta de que los mayores meses de consumo en Londres son enero y diciembre, debido a que son épocas de invierno, las personas utilizan más la calefacción, a diferencia de los meses de junio a agosto, que según la gráfica, son los de menor consumo porque puede que sea verano, y en Londres no hay artefactos que hagan lo contrario a la calefacción.

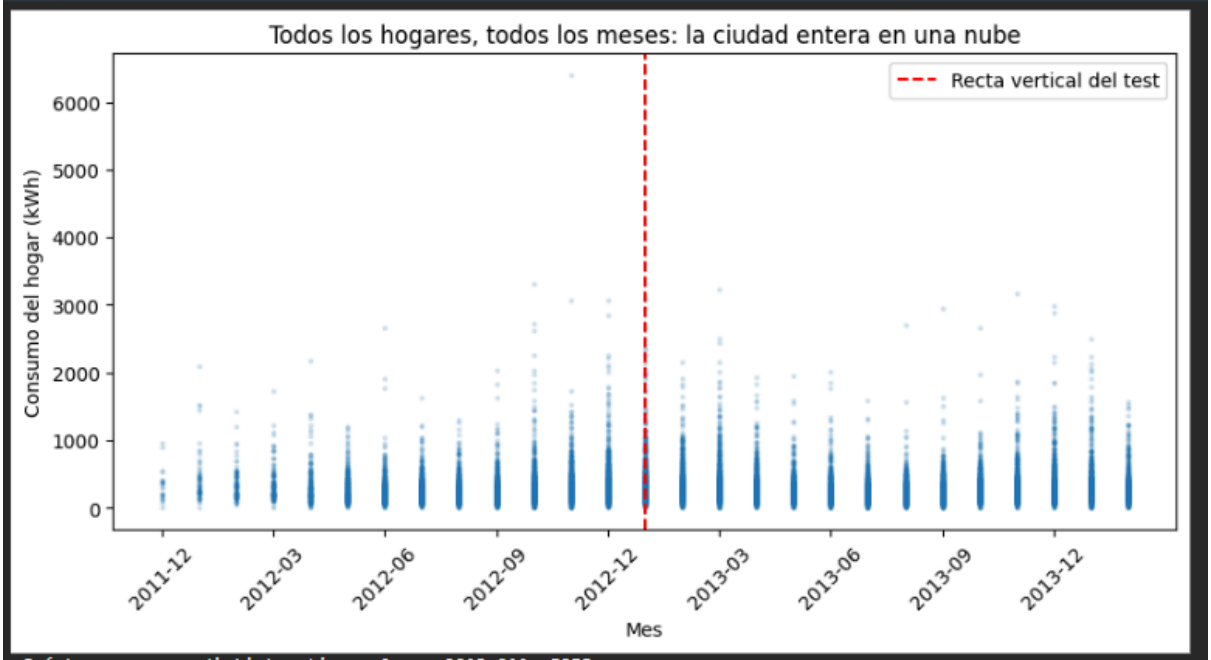
```python
# La nube completa (si hay muchos puntos, muestreamos 20.000 para que la gráfica respire).
muestra = df.sample(min(20000, len(df)), random_state=1)

meses = sorted(df["mes"].unique())
pos = {m: i for i, m in enumerate(meses)}   # posición de cada mes en el eje x

plt.figure(figsize=(10, 4.5))
plt.scatter(muestra["mes"].map(pos), muestra["kwh"], s=3, alpha=0.15)
plt.xticks(range(0, len(meses), 3), meses[::3], rotation=45)
plt.xlabel("Mes")
plt.ylabel("Consumo del hogar (kWh)")
plt.title("Todos los hogares, todos los meses: la ciudad entera en una nube")

# La recta vertical del test, dibujada sobre un mes cualquiera:
plt.axvline(pos[meses[len(meses)//2]], color="red", linewidth=1.5, linestyle="--",
            label="Recta vertical del test")
plt.legend()
plt.show()

un_mes = meses[len(meses)//2]
print(f"¿Cuántos consumos distintos tiene el mes {un_mes}?:",
      df[df['mes'] == un_mes]['kwh'].nunique())
```




```
¿Cuántos consumos distintos tiene el mes 2013-01?: 5258
```

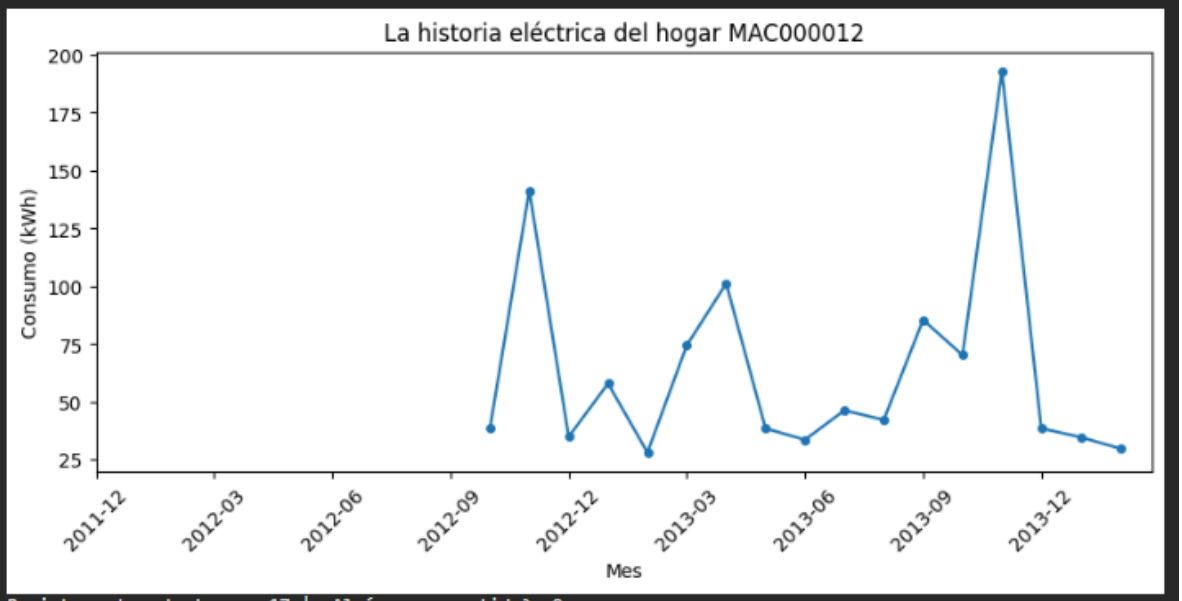
La gráfica da a entender que el consumo no es función del mes, ya que cada mes no tiene un valor único de Kwh, ya que cada mes es una columna de muchos puntos.

```python
# 🖍️ Ejercicio 4 — Un solo hogar.
# Elijan un código de hogar (mírenlos con df["hogar"].unique()[:10]) y grafiquen SU serie:
# mes en el eje x, kwh en el eje y. ¿Cuántos puntos atraviesa ahora una recta vertical?

hogar_elegido = df["hogar"].unique()[10]   # ← cambien por el hogar que quieran
serie = df[df["hogar"] == hogar_elegido].sort_values("mes")

plt.figure(figsize=(10, 4))
plt.plot(serie["mes"].map(pos), serie["kwh"], marker="o", markersize=4)
plt.xticks(range(0, len(meses), 3), meses[::3], rotation=45)
plt.xlabel("Mes")
plt.ylabel("Consumo (kWh)")
plt.title(f"La historia eléctrica del hogar {hogar_elegido}")
plt.show()

print("Registros de este hogar:", len(serie),
      "| ¿Algún mes repetido?:", serie["mes"].duplicated().sum())
```




```
Registros de este hogar: 17 | ¿Algún mes repetido?: 0
```

Una recta vertical atraviesa un punto o ninguno, pero no más de uno, además, con esta gráfica podemos ver que no hay meses repetidos, por lo que aparecen solo una vez, así que a cada mes le corresponde un único valor de Kwh, por lo que el consumo del hogar es función del mes.

Tarea puente:

```
1 !ls

agregar_londres.py  london_energy.csv  sample_data
```

```
1 !gunzip london_energy.csv.gz
```

```
1 !python agregar_londres.py

Hogares: 5556 | Filas: 113440 (descartadas por mes incompleto: 5038)
Rango: 2011-12 a 2014-02 | Mediana kWh/mes: 244.8
Escrito: londres_mensual.csv
```

El !ls verifica que los archivos si se subieron en el documento de colab, el siguiente comando permite cambiar de nombre a london_energy.csv.gz ya que estaba comprimido con gzip, esto lo cambia a london_energy.csv para que no ocurran errores con el archivo de kaggle, y con esto se puede ver que al correr agregar_londres, los valores son exactamente iguales al CSV original del curso.

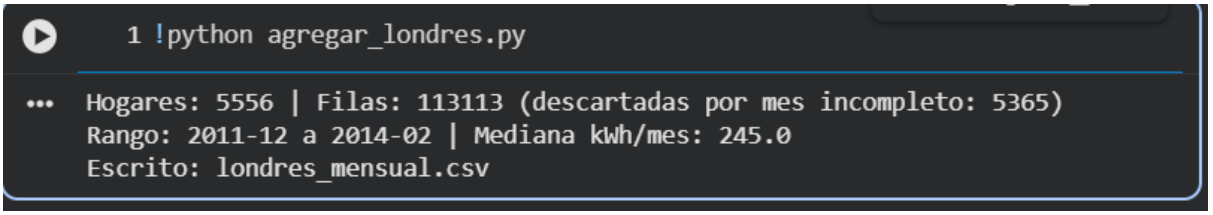
Regla de los 25 dias:
Suponemos que el profesor descarto los meses con menos de 25 dias porque estos ya no serían tan útiles para calcular un promedio de consumo, debido a los valores tan debajo de los que sí cumplen la regla, afectarían demasiado al promedio negativamente, ¿Estamos de acuerdo?, por una parte sí, ya que esta medida si es así, nos parece adecuada pero creemos que se puede perfeccionar un poco más.
Intento 1:

```
1 !python agregar_londres.py

Hogares: 5556 | Filas: 100936 (descartadas por mes incompleto: 17542)
Rango: 2011-12 a 2014-01 | Mediana kWh/mes: 244.7
Escrito: londres_mensual.csv
```

Aquí probamos con 30 días, pero vimos graves inconvenientes, como los descartados, que aumentaron demasiado, principalmente por todas las veces que aparecería febrero, debido a que no está cumpliendo con la regla nueva propuesta.

Segundo intento (definitivo):



Probando con 27 dias, lo más apropiado para nosotros, ya que no se está excluyendo febrero, y no se está tomando al 100% sus valores diarios, para no ser lo suficientemente estricto, esto se puede regular con los demás meses, parecido a los 25 días originales pero un poco más cercano a los posibles valores exactos.

El misterio de febrero:
La calefacción no falla en febrero, simplemente al tener menos días, el consumo va a ser menor.
Cambiando la variable para que la comparación sea justa, proponemos que esta sea un porcentaje en concreto para todos los meses, es decir, si en enero que hay 31 días, tomo el 85% - 90% de sus días, ya no va a haber tanta diferencia entre otros meses, especialmente con febrero.